📘 Section 6: Forensic Explainability & Interpretability Engine

📑 Table of Contents

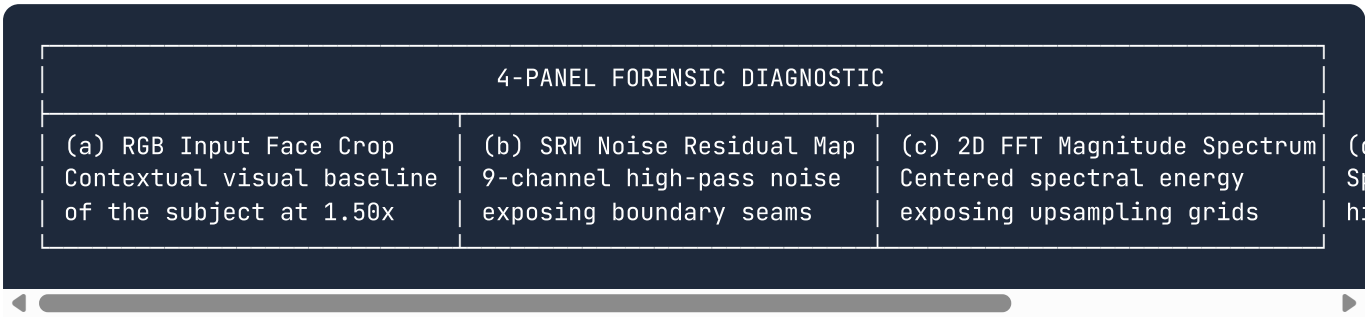
1. Overview & The Forensic Imperative for Explainability

In digital media forensics, cybersecurity, legal proceedings, and journalistic verification, a single classification number (e.g., $p = 0.99$) is insufficient.

Investigators and analysts require **evidence-based explanations**: - **Where** in the facial region did the neural network detect synthetic manipulation? - **What physical anomalies** triggered the classification (e.g., mouth

reenactment boundary, eye warping, or frequency grid artifacts)? - Is the model making a decision based on genuine forgery artifacts or spurious background shortcuts?

The explainability engine in `src/utils/interpretability.py` resolves this by generating a comprehensive **4-Panel Diagnostic Visualization** on demand for any input face.



2. The 4-Panel Diagnostic Standard

The 4-panel diagnostic layout provides a complete, multi-domain overview of an individual face crop:

Panel	Domain	Technique	Forensic Purpose
Panel (a)	Spatial RGB	$1.50\times$ Expanded Face Crop	Provides high-resolution visual context, head pose, and facial expression.
Panel (b)	Spatial Noise	9-Channel SRM Filter Bank (Viridis)	Suppresses scene content and exposes physical noise step-discontinuities along facial seams.
Panel (c)	Frequency	Centered 2D Real FFT (Magma)	Visualizes periodic frequency anomalies, radial energy decay, and GAN upsampling spikes.
Panel (d)	Semantic Attention	ConvNeXt Stage 4 Grad-CAM	Pinpoints the exact anatomical regions (eyes, mouth, boundary) driving the classifier decision.

3. Grad-CAM for ConvNeXt Spatial Backbone

Implemented in `ConvNeXtGradCAM`.

3.1 Theoretical Foundations of Class Activation Mapping

Gradient-weighted Class Activation Mapping (Grad-CAM) (Selvaraju et al., ICCV 2017) visualizes the spatial regions of an image that contribute most positively to a specific classification score z. - In ConvNeXt-Small, the final convolutional feature maps in `spatial_backbone[-1]` retain spatial geometry (8×8 grid for 256×256 inputs) while encoding rich semantic representations across $K = 768$ channels.

3.2 PyTorch Hook Mechanics (`register_forward_hook` & `backward_hook`)

In PyTorch, intermediate activations and gradients are discarded after backpropagation to save memory. In `ConvNeXtGradCAM.__init__`, persistent execution hooks are registered on the target layer:

```
target_layer = self.model.spatial_backbone[-1]
# Hook 1: Intercepts forward activation tensors A^k
self.forward_handle = target_layer.register_forward_hook(self._save_feature_maps)
# Hook 2: Intercepts backward gradient tensors d(z) / d(A^k)
self.backward_handle = target_layer.register_full_backward_hook(self._save_gradients)
```

3.3 Mathematical Formulation of Gradient-Weighted Activation

1. **Forward Pass**: The input tensor $x \in \mathbb{R}^{1\times 3\times H\times W}$ passes through the network, outputting raw scalar logit z. The forward hook saves the activation maps:

$$\mathbf{A} \in \mathbb{R}^{768\times H'\times W'}$$

2. **Backward Pass**: Backpropagation computes the gradient of the scalar logit z with respect to each activation map:

$$\mathbf{G}^k = \frac{\partial z}{\partial \mathbf{A}^k} \in \mathbb{R}^{H'\times W'}$$

3. **Neuron Importance Weights (α_k)**: Global average pooling computes the importance weight α_k for each of the 768 channels:

$$\alpha_k = \frac{1}{H'\cdot W'}\sum_{i=1}^{H'}\sum_{j=1}^{W'}\frac{\partial z}{\partial A^k_{i,j}}$$

4. **Weighted Linear Combination & Rectified Linear Activation**: The heatmap is computed as the weighted sum of all feature maps, passed through a **ReLU** non-linearity:

$$L_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_{k=1}^{768}\alpha_k\mathbf{A}^k\right) \in \mathbb{R}^{H'\times W'}$$

5. **Why ReLU is applied**: We are interested strictly in features that have a **positive influence** on the fake logit score (z). Features that decrease z are filtered out.

```
Target Layer Activations A^k [768, H', W']
                |
                ▼
Gradients d(z)/d(A^k) [768, H', W']
                |
                ▼
Global Average Pooling → Weights alpha_k [768]
                |
                ▼
Linear Combination: Sum( alpha_k * A^k )
                |
```

```
                ▼
   ReLU Non-Linearity → Raw Heatmap [H', W']
                |
                ▼
   Bilinear Upsample to (H, W) & Min-Max Normalize
```

3.4 Bilinear Upsampling & Min-Max Normalization

The coarse $H' \times W'$ (8×8) heatmap is normalized to $[0.0, 1.0]$ and upsampled to the full face crop resolution (512×512) using bilinear interpolation:

```
denom = float(cam.max() - cam.min())
if denom > 1e-6:
    cam = (cam - cam.min()) / denom
else:
    cam = np.zeros_like(cam)

cam_resized = cv2.resize(cam, (img_size, img_size), interpolation=cv2.INTER_LINEAR)
```

3.5 Memory-Safe Hook Lifecycle Management

PyTorch hooks consume GPU memory and can interfere with subsequent training if left attached. `ConvNeXtGradCAM.remove_hooks` safely detaches both hooks upon completion:

```
def remove_hooks(self) → None:
    self.forward_handle.remove()
    self.backward_handle.remove()
```

---

4. Panel B: SRM High-Pass Noise Residual Visualization

Implemented in `generate_face_diagnostics`.

4.1 Multi-Channel High-Pass Noise Extraction

The input face crop $x \in \mathbb{R}^{1\times3\times H\times W}$ is passed through the fixed SRM filter bank `model.srm(x)`, yielding a 9-channel noise residual tensor:

$$\mathbf{R}_{\text{SRM}} \in \mathbb{R}^{1\times9\times H\times W}$$

4.2 Mean Absolute Deviation Formulation

To compress the 9 high-pass channels into a single 2D spatial heatmap:

$$\bar{R}(x,y) = \frac{1}{9}\sum_{c=1}^{9} \left|R_c(x,y)\right| \in \mathbb{R}^{H\times W}$$

1. Min-max normalization scales the residual intensity to $[0.0, 1.0]$:

$$R_{\text{norm}}(x, y) = \frac{\bar{R}(x, y) - \min(\bar{R})}{\max(\bar{R}) - \min(\bar{R}) + \epsilon}$$

2. Quantized to an 8-bit integer matrix: $R_{\text{uint8}} = \text{uint8}(255.0 \cdot R_{\text{norm}})$.

4.3 False-Color Thermal Mapping with OpenCV Viridis

The grayscale noise intensity is mapped to the perceptual **Viridis colormap** (`cv2.COLORMAP_VIRIDIS`):

```
srm_colored = cv2.applyColorMap(srm_uint8, cv2.COLORMAP_VIRIDIS)
srm_rgb = cv2.cvtColor(srm_colored, cv2.COLOR_BGR2RGB)
```

- **Dark Purple / Blue**: Zero noise residual (smooth, natural facial areas).
- **Bright Green / Yellow**: High noise residual (sharp physical discontinuities, synthetic blending seams).

4.4 Forensic Interpretation of Boundary Seams

In authentic faces, the Viridis noise residual map displays uniform, low-intensity texture across skin surfaces. In deepfakes: - A distinct **bright halo ring** appears along the perimeter of the face (jawline, hairline), clearly marking where the autoencoder face mask was pasted onto the target body.

---

5. Panel C: Centered 2D Real FFT Log-Magnitude Spectrum

Implemented in `generate_face_diagnostics` .

5.1 Spectral Magnitude Extraction Across Channels

The combined 10-channel noise residual ($9\ \text{SRM} + 1\ \text{Bayar}$) passes through `model.fft()` , which extracts 10 log-magnitude channels: \mathbf{M} = \text{freq_maps}[0, 0:10] \in \mathbb{R}^{10 \times H \times W}

5.2 Radial Frequency Profiling & Dynamic Normalization

1. Compute average spectral magnitude across all 10 residual channels:

$$\bar{M}(u, v) = \frac{1}{10} \sum_{c=1}^{10} M_c(u, v) \in \mathbb{R}^{H \times W}$$

2. Apply `np.fft.fftshift` to position zero-frequency (DC) at the exact center $(H/2, W/2)$.

3. Min-max normalize to $[0, 255]$:

$$M_{\text{uint8}} = \text{uint8}\left(255.0 \cdot \frac{\bar{M}_{\text{centered}} - \min(\bar{M})}{\max(\bar{M}) - \min(\bar{M}) + \epsilon}\right)$$

5.3 False-Color Thermal Mapping with OpenCV Magma

The centered frequency spectrum is rendered using the **Magma colormap** (`cv2.COLORMAP_MAGMA`):

```
fft_colored = cv2.applyColorMap(fft_uint8, cv2.COLORMAP_MAGMA)
fft_rgb = cv2.cvtColor(fft_colored, cv2.COLOR_BGR2RGB)
```

- **Center Glow**: Low-frequency energy (overall facial shape).
- **Radial Outer Rings**: High-frequency energy decay.

5.4 Forensic Interpretation of GAN Checkerboard Spikes

- **Authentic Faces**: Exhibit smooth, isotropic $1/f^p$ radial power decay from the center outward to the corners.
- **Deepfake Faces**: Display distinct, periodic **geometric bright spots and cross-hatches** in high-frequency regions, caused by transposed convolution upsampling and checkerboard deconvolution grids.

---

6. The 4-Panel Visualization Pipeline (`visualize_attention_maps.py`)

Implemented in `scripts/visualize_attention_maps.py` .

6.1 End-to-End Diagnostic Generation

The script `visualize_attention_maps.py` evaluates a dataset of face crops and produces publication-ready multi-panel diagnostics:

```
# 1. Extract 4-panel diagnostic layers
diagnostics = generate_face_diagnostics(model, face_rgb, device=device, temperature=temp)

# 2. Extract active gating weight g
with torch.no_grad():
    f_s = model.spatial_fc(model.spatial_pool(model.spatial_norm(model.spatial_backbone(norm_
    f_f = model.freq_fc(model.freq_conv(model.fft(torch.cat([model.srm(img_t), model.bayar(i
    gate_val = float(model.gate_fc(torch.cat([f_s, f_f], dim=1)).mean().cpu().numpy())
```

6.2 Publication-Ready 300 DPI Matplotlib Rendering

Each figure is rendered using Matplotlib at **300 DPI** with a dark theme: - Subplot 1: (a) Input Face Crop (RGB) - Subplot 2: (b) SRM Noise Residuals (Viridis) - Subplot 3: (c) 2D FFT Magnitude Spectrum (Magma,

displaying active Gating Weight g) - Subplot 4: (d) ConvNeXt Grad-CAM Spatial Heatmap Overlay - Super Title: `Deepfake Verdict: FAKE | Prob: 0.9998 | Logit: +24.12 | Gate: 0.207`

Figures are saved to `figures/attention_maps/` for documentation, peer review, and forensic auditing.

---

7. Code Walkthrough & Reference

Component / Function	File Reference	Primary Responsibility
`ConvNeXtGradCAM`	`src/utils/interpretability.py#L17-L65`	Registers PyTorch forward/backward hooks, computes gradient-weighted activation maps, and upsamples heatmaps.
`generate_face_diagnostics`	`src/utils/interpretability.py#L67-L124`	Computes 4 diagnostic representations (RGB, SRM Viridis, FFT Magma, Grad-CAM).
`visualize_attention_maps.py`	`scripts/visualize_attention_maps.py`	Batch evaluation script rendering 300 DPI multi-panel figures for dataset inspection.

This document serves as the permanent reference for Section 6 (Forensic Explainability & Interpretability) of the Deepfake Detection Engine.